



UNIVERSIDAD DE
COSTA RICA
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Laboratorio 13: Proyecto Full-Stack Autónomo con Autenticación JWT y Formularios Reactivos ("MedPharm Express")

Resumen

Este laboratorio constituye la evaluación práctica integral de cierre previa al Examen Parcial I. Su objetivo principal es simular las condiciones reales de desarrollo autónomo que enfrentará en la prueba sumativa, exigiendo la construcción integral desde cero de una arquitectura de pila completa (Full-Stack).

Usted desarrollará la plataforma **MedPharm Express**, un sistema de gestión de recetas médicas y despacho de farmacia que integra persistencia en **H2 Database**, seguridad con **Spring Security y Tokens JWT**, servicios RESTful en **Spring Boot 3**, y un cliente SPA en **Angular 19 Standalone** estructurado con **Formularios Reactivos**, **Angular Signals** e **Interceptores HTTP**.

Metadatos del Laboratorio

- **Tiempo estimado de desarrollo:** 4.0 horas.
- **Herramientas requeridas:** Java 17/21+, Spring Boot 3, H2 Database (en memoria), Angular CLI 18/19+, Git, GitHub.

Metas de Aprendizaje:

1. Configurar la persistencia relacional en H2 Database y la arquitectura de seguridad con Spring Security + JWT en Spring Boot.
2. Desarrollar formularios reactivos complejos (FormGroup y FormArray), validaciones personalizadas, interceptores HTTP y guardias de ruta en Angular.
3. Construir una API RESTful robusta con manejo centralizado de excepciones (RFC 7807) y consumo reactivo mediante Angular Signals.

Contexto de Negocio: MedPharm Express

La red hospitalaria requiere automatizar el módulo de farmacia **MedPharm Express**. El sistema debe permitir que los médicos autenticados emitan recetas digitales compuestas por múltiples medicamentos, y que el personal farmacéutico consulte, filtre y actualice el estado de despacho de cada receta.

Usted deberá crear dos proyectos independientes desde cero:

- **Backend:** `medpharm-backend` (Spring Boot 3 en Java)
- **Frontend:** `medpharm-frontend` (Angular 19 Standalone)

Parte 1: Especificaciones Back-End (Spring Boot 3 + H2 + JWT)

El desarrollo del servidor backend debe realizarse de forma autónoma siguiendo los estándares de arquitectura en capas.

1. Configuración de Base de Datos H2 (`application.properties`)

Configure el archivo `src/main/resources/application.properties` para inicializar la base de datos H2 en memoria mediante los scripts DDL y DML provistos en la carpeta `resources/db/`:

```
spring.datasource.url=jdbc:h2:mem:medpharmdb;DB_CLOSE_DELAY=-1
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

# Desactivar autogeneracion para usar scripts SQL
spring.jpa.hibernate.ddl-auto=none
spring.h2.console.enabled=true
spring.sql.init.mode=always

# Clave secreta y expiracion para JWT
jwt.secret=MedPharmExpressSecretKeyForJWTAuthTokenGeneration2026Secured
jwt.expirationMs=86400000
```

Nota de Archivos SQL: Copie el contenido de los archivos `schema.sql` y `data.sql` ubicados en `Evaluaciones/Laboratorios/Laboratorio_13/resources/db/` hacia la carpeta `src/main/resources/` de su proyecto Spring Boot.

2. Mapeo Objeto-Relacional JPA (`com.medpharm.model`)

Implemente las entidades del dominio respetando los tipos de datos y relaciones:

- `[ ] Usuario.java`: Representa al personal del sistema (médicos y farmacéuticos). Campos: `id` (Long, PK), `username` (String, Unique), `password` (String), `nombreCompleto` (String), `rol` (String: MEDICO, FARMACEUTICO).
- `[ ] Medicamento.java`: Catálogo de medicinas. Campos: `id` (Long, PK), `codigo` (String, Unique), `nombre` (String), `stock` (Integer), `precioUnitario` (BigDecimal).

- [] **RecetaMedica.java**: Encabezado de la receta. Campos: id (Long, PK), codigoReceta (String, Unique), pacienteNombre (String), medico (@ManyToOne hacia Usuario), estado (String: PENDIENTE, DESPACHADA, CANCELADA), fechaEmision (LocalDateTime), detalles (@OneToMany hacia DetalleReceta).
- [] **DetalleReceta.java**: Renglones del medicamento prescrito. Campos: id (Long, PK), receta (@ManyToOne hacia RecetaMedica), medicamento (@ManyToOne hacia Medicamento), cantidad (Integer), dosisIndicada (String).

3. Seguridad y Autenticación JWT (`com.medpharm.security`)

Implemente la seguridad stateless mediante Tokens JWT:

- [] **Componente JwtUtils**: Métodos para generar token signed con HMAC-SHA256, obtener username desde el token y validar su firma/expiración.
- [] **Filtro AuthTokenFilter**: Interceptor Spring Security que extrae la cabecera Authorization: Bearer <token>, valida el JWT y establece el contexto de seguridad.
- [] **DTOs de Autenticación (`com.medpharm.dto`)**: LoginRequestDTO (username, password) y AuthResponseDTO (token, username, rol).
- [] **Controlador AuthController**: Endpoint POST `/api/v1/auth/login` que autentica las credenciales recibidas y retorna la firma JWT.

4. Capa de Servicios y API RESTful (`com.medpharm.service y controller`)

[] **RecetaService & RecetaController (`/api/v1/recetas`)**:

- GET `/api/v1/recetas`: Retorna la lista de recetas registradas.
- GET `/api/v1/recetas/estado/{estado}`: Filtra recetas por estado.
- POST `/api/v1/recetas`: Registra una nueva receta con sus detalles (valida disponibilidad de stock).
- PATCH `/api/v1/recetas/{id}/estado`: Modifica el estado de la receta (DESPACHADA, CANCELADA).

[] **MedicamentoController (`/api/v1/medicamentos`)**:

- `GET /api/v1/medicamentos`: Consulta el catálogo de medicamentos para poblar selectores.
- [] Habilite la política CORS global o a nivel de controlador para permitir peticiones desde `http://localhost:4200` incluyendo la cabecera `Authorization`.

Parte 2: Especificaciones Front-End (Angular 19 Standalone)

Inicialice el cliente SPA mediante la CLI de Angular (`ng new medpharm-frontend --standalone`).

1. Servicio de Autenticación e Interceptor HTTP

- [] **AuthService** (`src/app/services/auth.service.ts`): Gestiona los métodos `login(credentials)`, `logout()`, `getToken()` y almacena el token JWT en `localStorage`. Expone la sesión mediante `signal` o `BehaviorSubject`.
- [] **Interceptor JWT** (`src/app/interceptors/auth.interceptor.ts`): `HttpInterceptorFn` que clona cada petición saliente hacia la API e inyecta la cabecera `Authorization: Bearer <token>` si el usuario posee una sesión activa.
- [] **Guardia de Ruta** (`src/app/guards/auth.guard.ts`): Protege las rutas internas redirigiendo al usuario hacia `/login` en caso de no contar con token.

2. Componente de Inicio de Sesión (`LoginComponent`)

- [] **Ruta**: `/login`
- [] Implemente un **Formulario Reactivo** (`FormGroup`) con los campos `username` y `password`.
 - [] **Validaciones Requeridas**:
 - `username`: Requerido.
 - `password`: Requerido, longitud mínima de 6 caracteres.
- [] Al autenticar exitosamente, almacene el JWT y redirija al usuario hacia la ruta `/recetas`.

3. Componente Dashboard de Recetas (`RecetasListComponent`)

- [] **Ruta:** `/recetas` (Ruta protegida por `AuthGuard`).
- [] **Reactividad con Signals:** Almacene el listado de recetas mediante `signal()` y utilice `computed()` para filtrar dinámicamente según el estado seleccionado (`TODAS`, `PENDIENTE`, `DESPACHADA`).
- [] Presente una tabla responsiva con badges visuales de color y botones interactivos para cambiar el estado de la receta mediante peticiones HTTP `PATCH`.

4. Componente Formulario de Creación (`RecetaFormComponent`)

- [] **Ruta:** `/nueva-receta` (Ruta protegida por `AuthGuard`).
- [] Implemente un **Formulario Reactivo Anidado** con `FormGroup` y `FormArray` para agregar múltiples renglones de medicamentos a la receta.

[] **Validaciones Requeridas:**
 - [] `pacienteNombre`: Requerido, mínimo 5 caracteres.
 - [] `medicamentoId`: Requerido.
 - [] `cantidad`: Requerido. **Validador Personalizado (`positivoValidator`)**: Debe comprobar que la cantidad prescrita sea un número entero estrictamente mayor a 0.
- [] Consuma el servicio Angular para enviar el payload JSON al servidor y retornar al dashboard.

Parte 3: Análisis y Depuración de Errores

Durante el desarrollo de aplicaciones con autenticación basada en JWT e interceptores HTTP, es sumamente común enfrentar bloqueos por políticas CORS Preflight (solicitudes `OPTIONS`) o errores de autorización `401 Unauthorized`.

Como parte de la evaluación de depuración autónoma, usted deberá realizar la siguiente prueba:

- [] Intente consumir el endpoint protegido GET /api/v1/recetas desde Angular **sin** registrar el `authInterceptor` en `app.config.ts`.
- [] Verifique la falla en la pestaña *Network* de la consola del desarrollador del navegador (código de estado HTTP 401 / 403).
- [] Tome una captura de pantalla del error devuelto por la consola y guárdela en su repositorio bajo la ruta `docs/error_jwt_401.png`.
- [] Documente en el archivo `README.md` de su proyecto la explicación técnica de por qué el servidor rechazó la petición y cómo el interceptor HTTP resuelve el envío del encabezado `Authorization: Bearer <token>`.

Parte 4: Reto Autónomo y Entrega

Su objetivo final es entregar una solución funcional que demuestre el dominio de Spring Boot 3 y Angular 19 Standalone de cara al Examen Parcial I.

Rúbrica de Evaluación (100 Puntos)

Criterio Técnico	Puntos	Detalles
Persistencia H2 & Mapeo JPA	20 pts	Configuración H2, entidades JPA con relaciones @ManyToOne/@OneToMany y scripts SQL.
Seguridad Spring Boot & JWT	20 pts	Generación y validación de tokens JWT, filtro de seguridad y endpoint /login.

API RESTful & Servicios	15 pts	Endpoints /recetas y /medicamentos, DTOs y manejo centralizado de excepciones.
Auth Angular & Interceptor	15 pts	Servicio AuthService, authInterceptor HTTP y protección con AuthGuard.
Formularios Reactivos Angular	20 pts	FormGroup, FormArray, validadores nativos y validador personalizado (positivoValidator).
Depuración y Git Semántico	10 pts	Documentación de error 401 en README.md, captura docs/error_jwt_401.png y commits semánticos.

Instrucciones de Entrega

1. Inicialice un repositorio Git local en la raíz de su espacio de trabajo.
2. Realice commits semánticos periódicos durante la sesión (mínimo 5 commits, ej.:
feat(backend): setup H2 database and JPA entities, feat(angular): add auth interceptor and login component).
3. Cree un repositorio público en GitHub con la estructura estándar de carpetas:
IF0009-Lab13-carnet.
4. Suba su código fuente completo junto con el informe de depuración en el archivo README.md.
5. Registre la URL de su repositorio en la plataforma virtual de aprendizaje antes del cierre de la sesión.

¡Éxito en la resolución autónoma de este laboratorio integrador!